



UNIVERSIDAD DEL BÍO-BÍO

CIMUBB
LABORATORIO DE SISTEMAS
AUTOMATIZADOS DE PRODUCCION
Facultad de Ingeniería

Proyecto 1: Asistencia digital

Integrantes :
Marcelo Paz
Bastián Rodríguez

Supervisor :
Luis Vera

27 de febrero de 2025

Índice de contenidos

1	Introducción	3
1.1	Situación actual	3
1.2	Objetivos del proyecto	3
1.2.1	Objetivo general	3
1.2.2	Objetivos específicos	3
2	Análisis y Diseño	4
2.1	Diseño lógico	4
2.1.1	Modelo de entidad-relación	4
2.1.2	Modelo relacional	4
2.1.3	Diagrama de contexto	5
2.1.4	Diagrama de nivel superior	5
2.1.5	Procedimientos administrativos	6
2.2	Diseño físico	7
2.2.1	Arquitectura del sistema	7
3	Documentación de Hardware	8
3.1	Raspberry Pi 4	8
3.2	Pantalla táctil de 7 pulgadas [Se desconoce el tipo de pantalla]	8
3.3	Logitech HD Webcam C270	9
3.4	Sensor QR	9
4	Documentación desarrollo del Software	10
4.1	Etapas 0	10
4.2	Etapas 1	10
4.3	Etapas 2	10
4.4	Etapas 3 (Sin pruebas)	10
4.5	Etapas 4 (Sin pruebas)	11
5	Implementación	11
5.1	Software registro de asistencia	11
5.1.1	Librerías	11
5.2	Software consultor	12
5.2.1	Librerías	12
5.2.2	Puesta en marcha (MANUAL)	12
5.2.3	Puesta en marcha (AUTOMÁTICA)	17
5.2.4	Capturas de la interfaz	18

Índice de figuras

1	Modelo de entidad-relación	4
2	Modelo relacional	4
3	Diagrama de contexto	5
4	Diagrama de nivel superior	5
5	Procedimientos administrativos	6
6	Arquitectura del sistema	7
7	Raspberry Pi 4 [1]	8
8	Pantalla táctil de 7 pulgadas	8
9	Logitech HD Webcam C270 [3]	9
10	Sensor QR GM811 [2]	9
11	Requerimientos software consultor	12
12	Generate Random Strings and Numbers [4]	16
13	Interfaz del home	18
14	Interfaz del login	18
15	Interfaz del dashboard	19
16	Interfaz del profile	19
17	Interfaz del register admin	20
18	Interfaz del register user	20
19	Componente Navbar	21
20	Componente Notify	21

Índice de tablas

1. Introducción

1.1. Situación actual

Respecto al estudio realizado en el establecimiento, se observó un gran flujo de personas que hacen ingreso al Laboratorio utilizando un registro de asistencia manual. Algunas de estas personas son trabajadores de la institución, estudiantes en prácticas o invitados. Existen ocasiones especiales, como exposiciones de maquetas o presentaciones de nuevas tecnologías en desarrollo, en la cual ingresa personal externo a la institución.

1.2. Objetivos del proyecto

1.2.1. Objetivo general

- Construir un sistema de asistencia digital que permita conocer el ingreso y salida de los individuos dentro del laboratorio.
- Visualizar los datos registrados en un dashboard con información sobre el uso del espacio y las actividades principales.

1.2.2. Objetivos específicos

- Recepción de datos de individuos.
- Validación de datos utilizando un API proporcionada por la Universidad del Bío-Bío.
- Almacenamiento de datos registrados.
- Visualización de datos registrados.

2. Análisis y Diseño

2.1. Diseño lógico

2.1.1. Modelo de entidad-relación

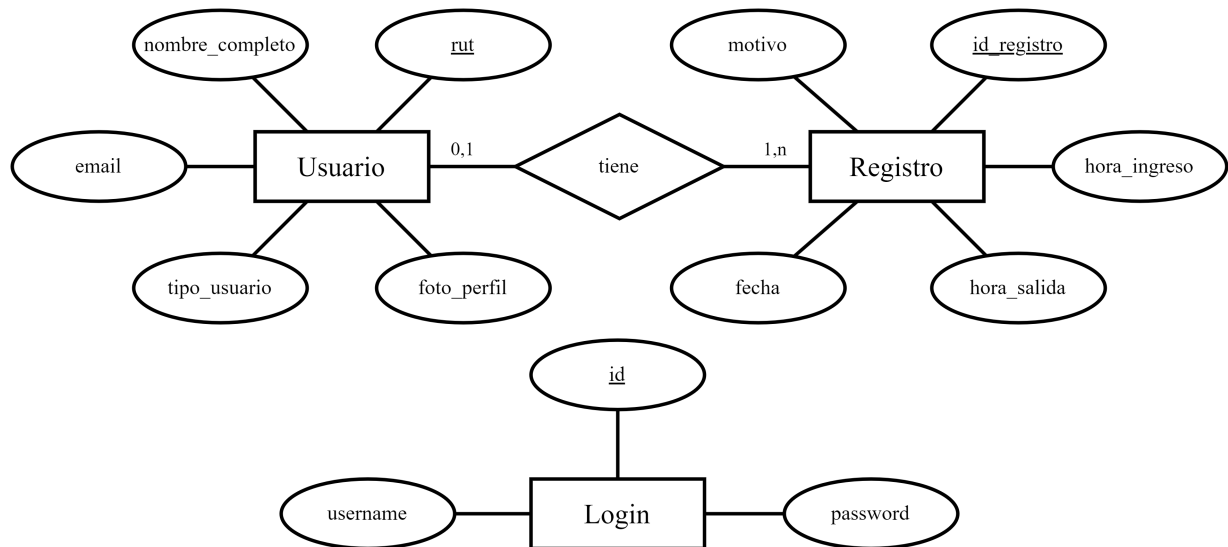


Figura 1: Modelo de entidad-relación

2.1.2. Modelo relacional

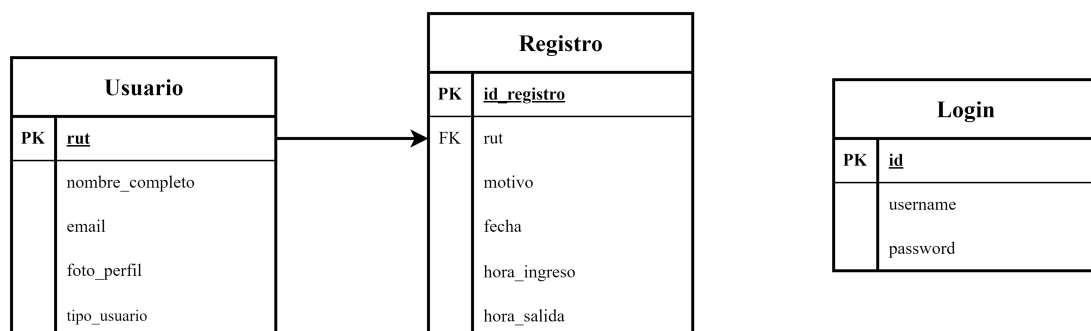


Figura 2: Modelo relacional

2.1.3. Diagrama de contexto

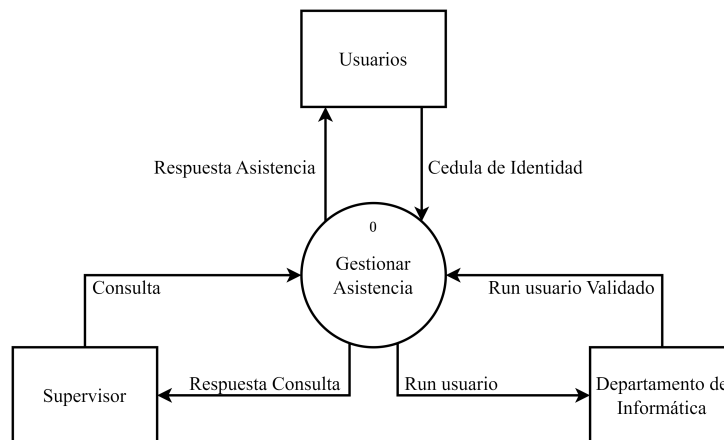


Figura 3: Diagrama de contexto

2.1.4. Diagrama de nivel superior

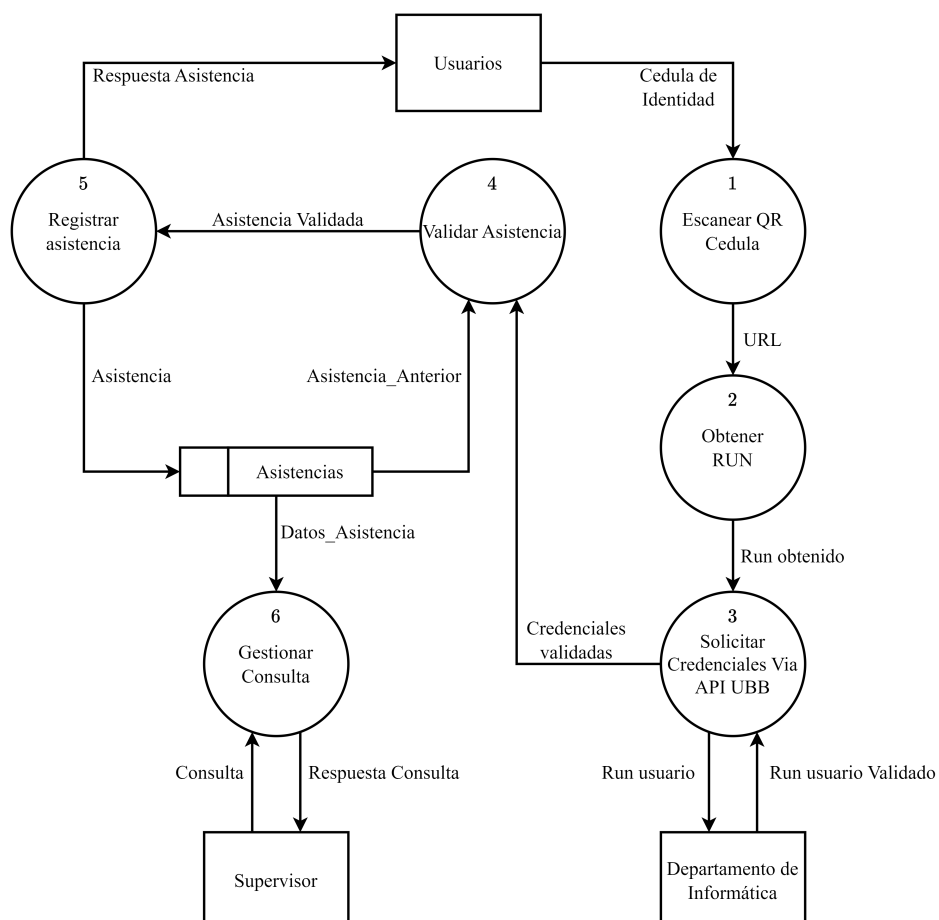


Figura 4: Diagrama de nivel superior

2.1.5. Procedimientos administrativos

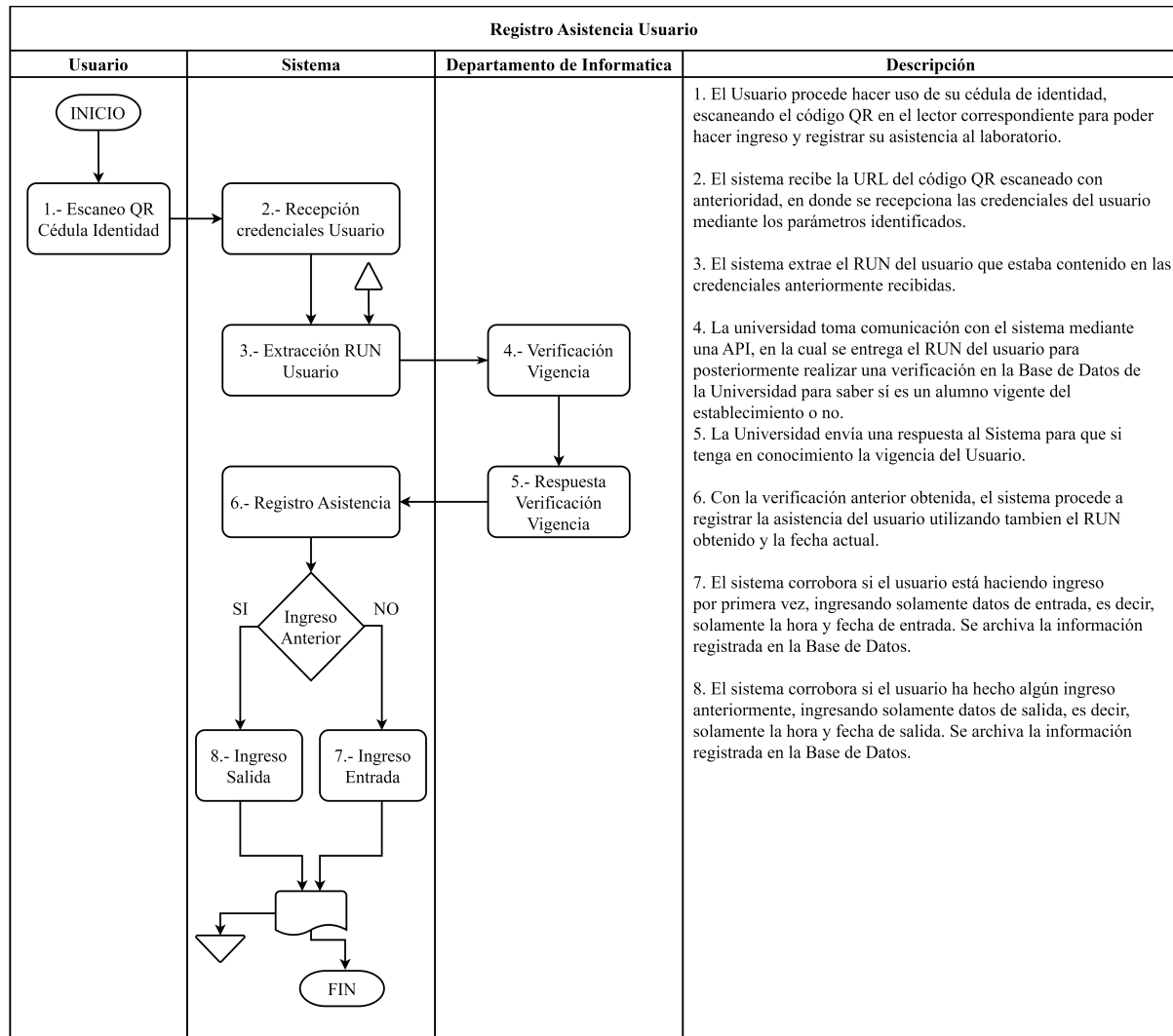


Figura 5: Procedimientos administrativos

2.2. Diseño físico

2.2.1. Arquitectura del sistema

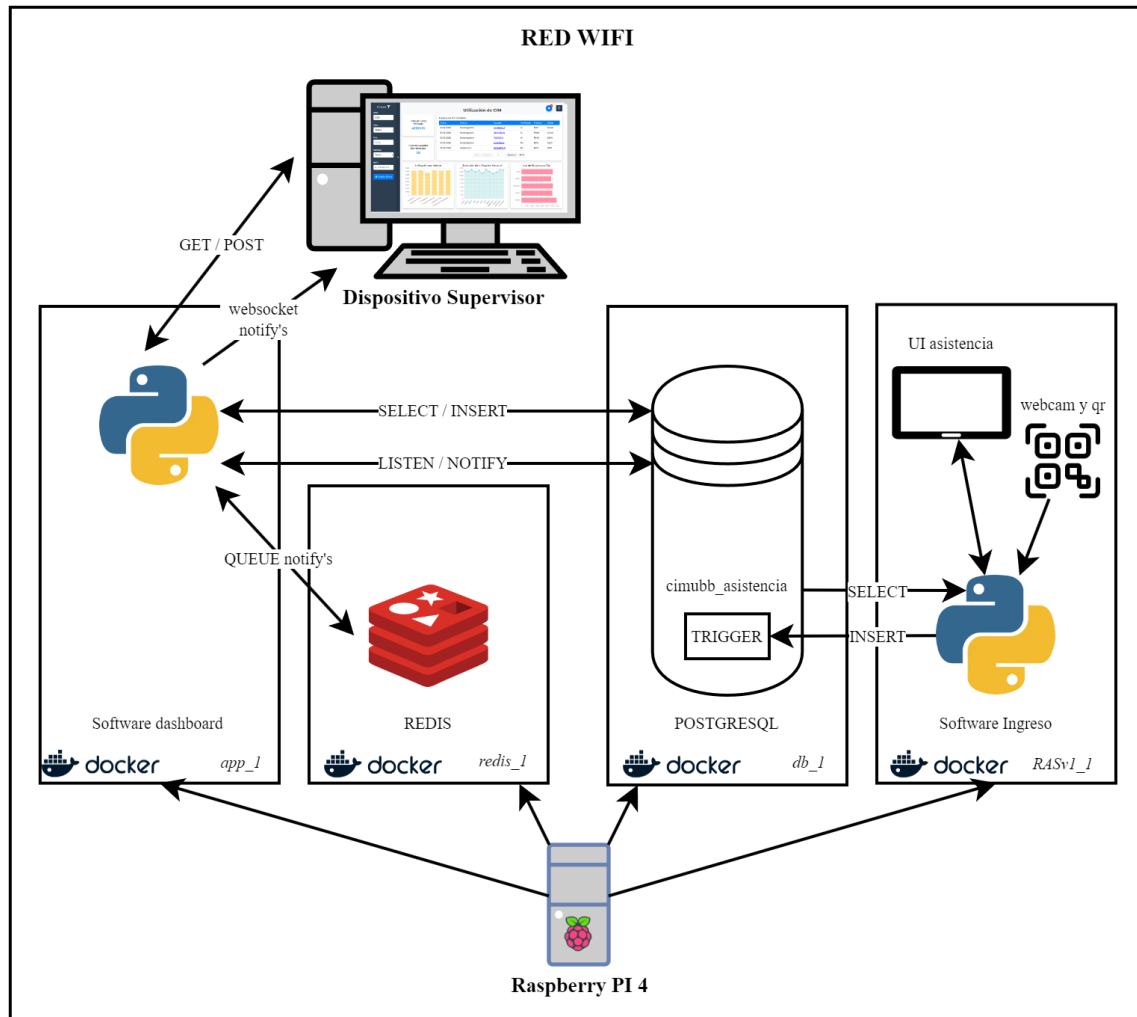


Figura 6: Arquitectura del sistema

Se decidió utilizar Docker para facilitar la implementación del software. En resumen, dentro de la raspberry tendremos 4 imagenes de Docker:

- **app_1**: La aplicación que controla el servicio web para la supervisión de los registros.
- **redis_1**: Se utiliza una imagen oficial de redis para el guardado de notificaciones temporales, pues si no tenemos activo un WebSocket se pierden las notificaciones, por lo que el usar Redis de forma temporal nos resuelve este problema al utilizar una Queue.
- **db_1**: Se utiliza una imagen oficial de postgres:17 para guardar nuestros datos y montar inicialmente la configuración.
- **RASv1_1**: La aplicación encargada de mostrar una GUI para el registro de asistencias del laboratorio.

3. Documentación de Hardware

3.1. Raspberry Pi 4

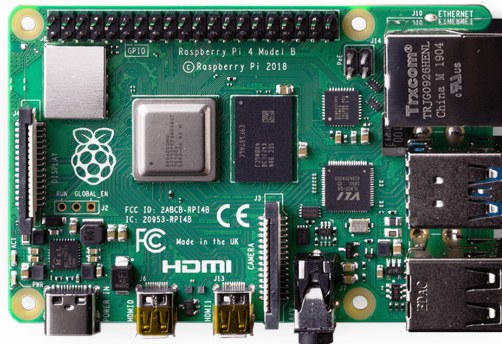


Figura 7: Raspberry Pi 4 [1]

La Raspberry Pi 4 Model B de 8GB de RAM es utilizada en el proyecto por su capacidad para manejar aplicaciones intensivas en procesamiento y datos, su conectividad avanzada, que incluye Wi-Fi y Ethernet, asegura la comunicación eficiente con servidores o sistemas remotos, facilitando el monitoreo y notificación del ingreso de personal al laboratorio.

3.2. Pantalla táctil de 7 pulgadas [Se desconoce el tipo de pantalla]



Figura 8: Pantalla táctil de 7 pulgadas

La pantalla táctil de 7 pulgadas es utilizada para mostrar el software de registro de asistencia, permitiendo a los usuarios ingresar su información personal y registrar su ingreso al laboratorio.

3.3. Logitech HD Webcam C270



Figura 9: Logitech HD Webcam C270 [3]

La Logitech HD Webcam C270 es utilizada para capturar la foto de perfil de las personas que se registran en el sistema.

3.4. Sensor QR



Figura 10: Sensor QR GM811 [2]

El sensor QR GM811 es utilizado para leer los códigos QR de las cédulas de identidad, de las personas que ingresan al laboratorio, permitiendo un registro rápido y eficiente de la asistencia.

Lo más importante de este sensor es que tiene un modo de POS (Point of Sale) que permite leer códigos QR de manera rápida y eficiente, lo que lo hace ideal para el proyecto.

4. Documentación desarrollo del Software

4.1. Etapa 0

Se elaboro de un informe para la asignatura Sistemas de Información, en el cual se desarrolló un prototipo de un sistema de asistencia digital para el Laboratorio de Sistemas Automatizados de Producción de la Universidad del Bío-Bío. El software inicialmente fue desarrollado de forma rudimentaria, pues lo importante del informe era llevar a cabo una idea, definir ciertos aspectos y realizar los diagramas de contexto, nivel superior y de procedimientos administrativos.

4.2. Etapa 1

En esta etapa se utilizo la idea de la Etapa 0 para desarrollarlo en la practica profesional II dentro del Laboratorio de Sistemas Automatizados de Producción de la Universidad del Bío-Bío.

Se utilizo inicialmente un dispositivo Windows con una cámara web para el registro de asistencia, el software analizaba la imagen con opencv y detectaba el código QR de la cédula de identidad, luego se enviaba la información a una base de datos en PostgreSQL.

4.3. Etapa 2

En esta etapa se utilizo una Raspberry Pi 4 con una cámara web, se utilizo el SO Raspbian y el mismo software de la etapa 1.

Se observo que la Raspberry Pi 4 tenia problemas al procesar la imagen de la cámara web, tanto en la detección del código QR como en la velocidad de actualización de la imagen en la interfaz gráfica. Al intentar resolver el problema, se crearon más problemáticas, pues se intento implementar la pantalla táctil que se comunicaba por GPIO, lo que genero problemáticas en la comunicación, dejando la pantalla en negro, por lo que se tuvo que reinstalar el SO.

Finalmente se decidió cambiar el software de registro de asistencia y utilizar un Sensor QR.

4.4. Etapa 3 (Sin pruebas)

Se desarrollo desde cero, el software de registro de asistencia, implementando el sensor QR. Lo malo es que mientras se descubria el funcionamiento del sensor, se configuro a un modo que solo aceptaba configuracion por comunicacion Serial, pero luego no era reconocido por ningun dispositivo. Lo que nos retraso en nuestro limitado tiempo.

El supervisor nos recomendo hacer algunos avances sin realizar pruebas para entregar al menos una idea del software a implementar, para completar la practica profesional II.

4.5. Etapa 4 (Sin pruebas)

Se desarrollo una aplicación web para la visualización de los datos, para ello se utilizo Python con FastAPI. Para el acceso a esta aplicación, el supervisor debe acceder a la IP de las raspberry por el puerto 8000, la aplicación cuenta con un ingreso por PWT, para la seguridad mínima del sistema.

Durante el desarrollo se fueron reformulando ciertas ideas, lo que llevo a algunas modificaciones de la BD, como por ejemplo la capacidad de notificar de un nuevo ingreso, la actualización automatica de la hora de salida si es que una persona no hubiera marcado y la actualización automatica de la zona horaria para invierno/verano.

Al final de esta etapa se decidió utilizar Docker para contener los diferentes módulos del SW, aparte de su facil implementación, pues con el docker compose se puede crear facilmente una DB segura y compacta.

5. Implementación

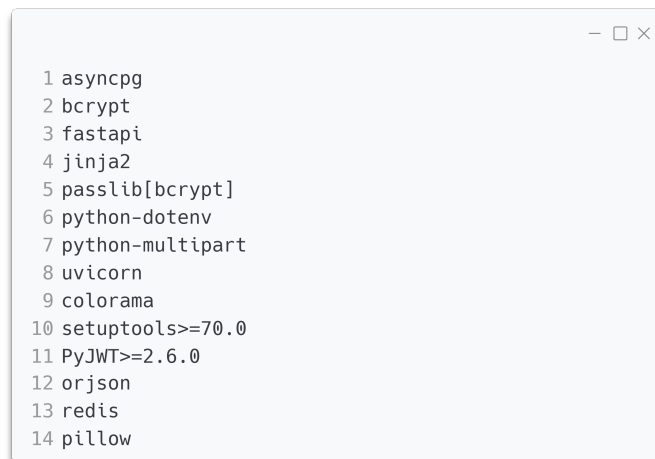
5.1. Software registro de asistencia

5.1.1. Librerías

5.2. Software consultor

Para el correcto funcionamiento de este software se debe tener en cuenta lo siguiente:

5.2.1. Librerías



```
1 asyncpg
2 bcrypt
3 fastapi
4 jinja2
5 passlib[bcrypt]
6 python-dotenv
7 python-multipart
8 uvicorn
9 colorama
10 setuptools>=70.0
11 PyJWT>=2.6.0
12 orjson
13 redis
14 pillow
```

Figura 11: Requerimientos software consultor

5.2.2. Puesta en marcha (MANUAL)

WINDOWS

1. Crear una base de datos en PostgreSQL con el nombre *cimubb_asistencia*.

- Abrir psql con las credenciales del usuario postgres.
- Crear la base de datos con el comando:

```
CREATE DATABASE cimubb_asistencia;
```

c) Nos conectamos a la BD creada:

```
\c cimubb_asistencia
```

Forma automática:

d) Ejecutamos el archivo init.sql, utilizando la ruta del archivo. Como cron no existe para windows las tareas de marcar salida y actualizar zona horaria no funcionarían:

```
\i ruta/al/repositorio\Asistencia-CIMUBB\Dashboard-CIMUBB\init.sql
```

Forma manual:**e) Creamos las tablas.**

```
CREATE TABLE Usuario (  
    rut VARCHAR(10) PRIMARY KEY NOT NULL,  
    nombre_completo VARCHAR(50) DEFAULT '',  
    email VARCHAR(50) DEFAULT '',  
    tipo_usuario VARCHAR(15) NOT NULL,  
    foto_perfil BYTEA DEFAULT NULL  
);  
  
CREATE TABLE Registro (  
    id_registro VARCHAR(10) PRIMARY KEY NOT NULL,  
    fecha DATE NOT NULL,  
    hora_ingreso TIME NOT NULL,  
    hora_salida TIME DEFAULT NULL,  
    motivo VARCHAR(50) NOT NULL,  
    rut VARCHAR(10) NOT NULL,  
    FOREIGN KEY (rut) REFERENCES Usuario(rut)  
);  
  
CREATE TABLE Login (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR UNIQUE NOT NULL,  
    password VARCHAR NOT NULL  
);
```

f) Habilitamos la extensión crypto e insertamos el login para el admin.

```
-- Extensión para almacenar contraseñas encriptadas  
CREATE EXTENSION IF NOT EXISTS pgcrypto;  
  
-- Insertar usuario administrador con contraseña encriptada  
INSERT INTO Login (username, password)  
VALUES ('admin', crypt('admin', gen_salt('bf')));
```

g) Habilitamos la extensión cron y creamos una función para marcar la salida automática a las 19:00 de los registros que se les olvido marcar la hora salida.

```
-- Habilitar pg_cron si no existe  
CREATE EXTENSION IF NOT EXISTS pg_cron;  
  
-- Crear la función para actualizar la hora de salida automáticamente  
CREATE OR REPLACE FUNCTION marcar_salida_automatica()  
RETURNS VOID AS $$  
BEGIN  
    UPDATE Registro  
    SET hora_salida = '19:00:00'  
    WHERE hora_salida IS NULL AND fecha = CURRENT_DATE;  
END;  
$$ LANGUAGE plpgsql;  
  
-- Programar la tarea para ejecutarse todos los días a las HH:MM  
SELECT cron.schedule(  
    'marcar_salida_diaria',  
    '00 22 * * *', -- MM HH DD MM DOW (https://crontab.guru/)  
    'SELECT marcar_salida_automatica();'  
);
```

- h) Creamos una función para actualizar la zona horaria, pues postgres trabaja con UTC.

```
CREATE OR REPLACE FUNCTION actualizar_horario_pgcron()
RETURNS VOID AS $$
DECLARE
    diferencia_horaria INT;
    nueva_hora INT;
    nueva_hora_cron INT;
BEGIN
    -- Detectar la diferencia horaria real de Chile con UTC
    SELECT EXTRACT(TIMEZONE_HOUR FROM NOW()) AT TIME ZONE 'America/Santiago') INTO
    diferencia_horaria;

    IF diferencia_horaria = -3 THEN
        nueva_hora := 22; -- Chile UTC-3 (verano)
        nueva_hora_cron := 3; -- Medianoche en Chile en UTC-3
    ELSE
        nueva_hora := 23; -- Chile UTC-4 (invierno)
        nueva_hora_cron := 4; -- Medianoche en Chile en UTC-4
    END IF;

    -- Eliminar el job anterior si existe
    PERFORM cron.unschedule('marcar_salida_diaria');
    PERFORM cron.unschedule('actualizar_horario_diario');

    -- Programar la tarea para actualizar la salida automática
    PERFORM cron.schedule(
        'marcar_salida_diaria',
        format('00 %s * * *', nueva_hora),
        'SELECT marcar_salida_automatica();'
    );

    -- Programar la tarea para actualizar el horario de ejecución
    PERFORM cron.schedule(
        'actualizar_horario_diario',
        format('0 %s * * *', nueva_hora_cron),
        'SELECT actualizar_horario_pgcron();'
    );
END;
$$ LANGUAGE plpgsql;

SELECT cron.schedule(
    'actualizar_horario_diario',
    '0 3 * * *', -- Ejecutar cada día a medianoche UTC CHILE
    'SELECT actualizar_horario_pgcron();'
);
```

*OBS: El paso g) y h) no funcionarían porque CRON no existe para Windows, por lo que estas funciones no aplican, pues el software fue desarrollado para raspberryPi.

- i) Creamos una función para habilitar un LISTEN / NOTIFY con un trigger al detectar un ingreso o salida.

```
CREATE OR REPLACE FUNCTION notify_registro_changes()
RETURNS TRIGGER AS $$
DECLARE
    payload TEXT;
BEGIN
    IF TG_OP = 'INSERT' THEN
        payload := NEW.rut || ' ingreso a las ' || TO_CHAR(NEW.hora_ingreso,
        'HH24:MI');
    ELSIF TG_OP = 'UPDATE' AND OLD.hora_salida IS DISTINCT FROM NEW.hora_salida
    THEN
        payload := NEW.rut || ' salio a las ' || TO_CHAR(NEW.hora_salida,
        'HH24:MI');
    ELSE
        RETURN NEW;
    END IF;

    PERFORM pg_notify('notify_channel', payload);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER registro_notify_trigger
AFTER INSERT OR UPDATE ON Registro
FOR EACH ROW
EXECUTE FUNCTION notify_registro_changes();
```

2. Clonamos el repositorio:

```
git clone https://github.com/CIMUBB/Asistencia-CIMUBB.git
```

3. Nos movemos a la carpeta del proyecto:

```
cd .\Dashboard-CIMUBB\
```

4. Creamos un entorno virtual de python y lo activamos:

```
python -m venv env
.\env\Scripts\activate.bat
```

5. Instalamos las dependencias:

```
pip install -r .\requirements.txt
```

6. Configuramos el archivo .env dentro de la carpeta Dashboard-CIMUBB con las siguientes variables:

```
SECRET_KEY=  
ACCESS_TOKEN_EXPIRE_MINUTES=30  
DB_HOST=  
DB_PORT=5432  
DB_NAME=cimubb_asistencia  
DB_USER=postgres  
DB_PASSWORD=
```

***OBS:** Para la generación del SECRET_KEY utilizaremos la página browserling[4]

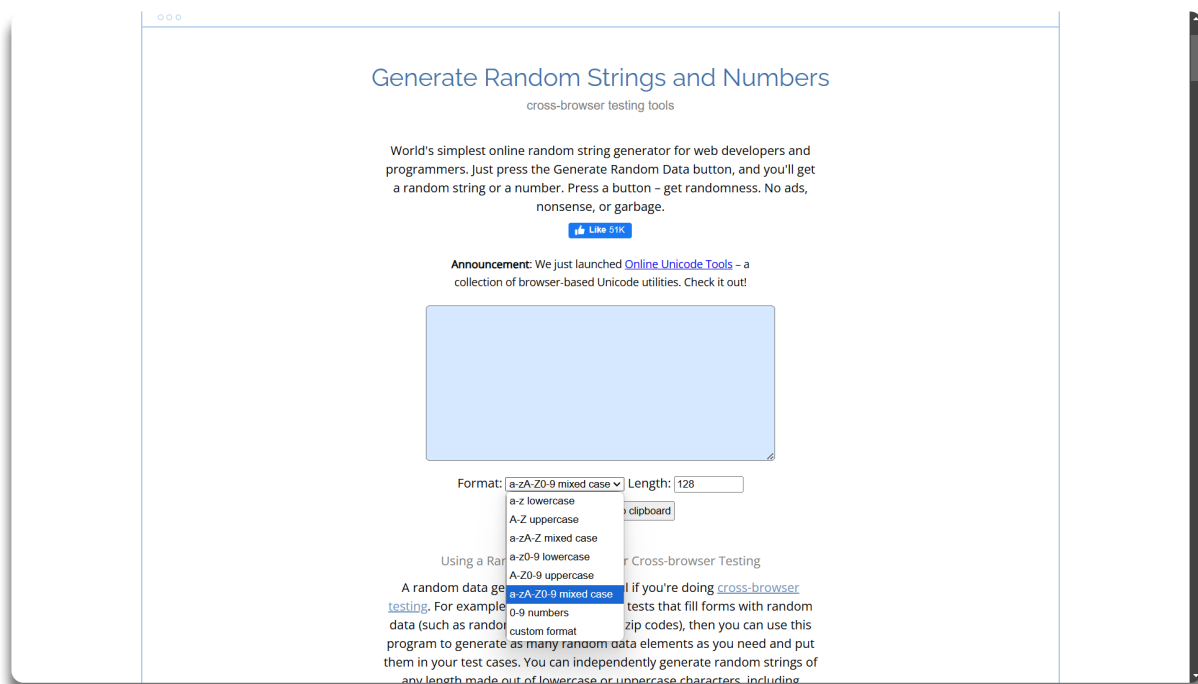


Figura 12: Generate Random Strings and Numbers [4]

7. Para finalizar ejecutamos el servidor de UVICORN:

```
uvicorn main.app:app --host 0.0.0.0 --port 8000 --workers 2
```

5.2.3. Puesta en marcha (AUTOMÁTICA)

DOCKER

1. Teniendo instalado docker, clonamos el repositorio:

```
git clone https://github.com/CIMUBB/Asistencia-CIMUBB.git
```

2. Nos movemos a la carpeta del proyecto:

```
cd .\Dashboard-CIMUBB\
```

5.2.4. Capturas de la interfaz

- <http://{IP}:8000/>



Figura 13: Interfaz del home

- <http://{IP}:8000/login>

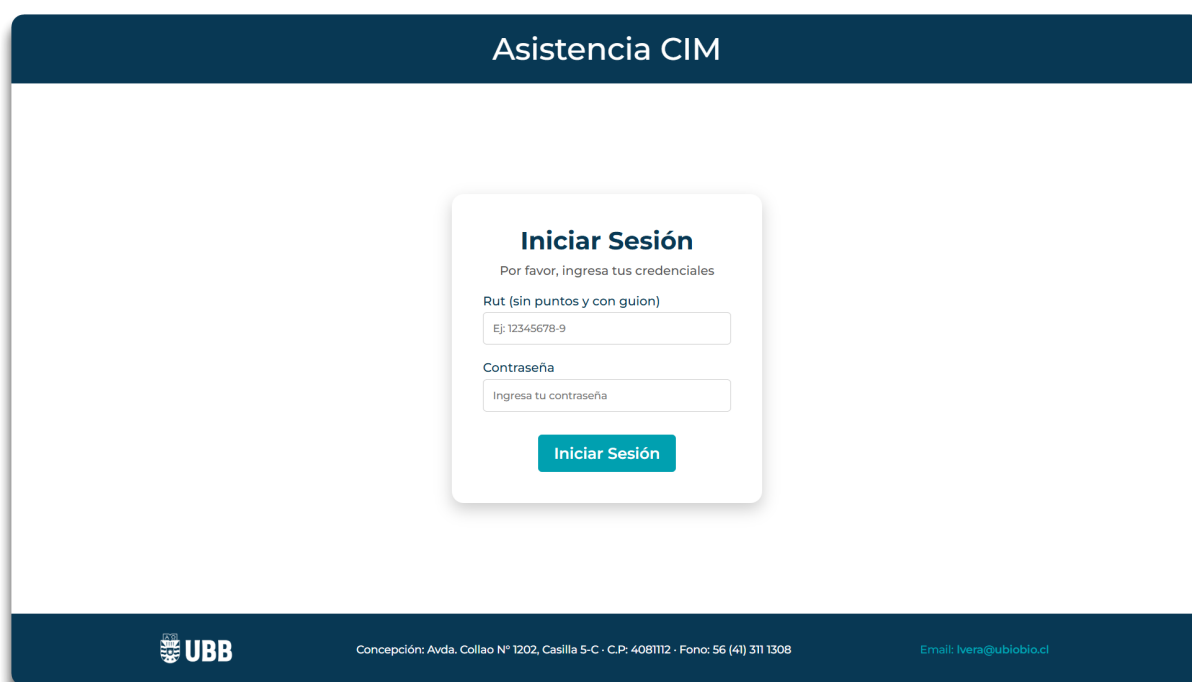


Figura 14: Interfaz del login

- <http://{IP}:8000/dashboard>

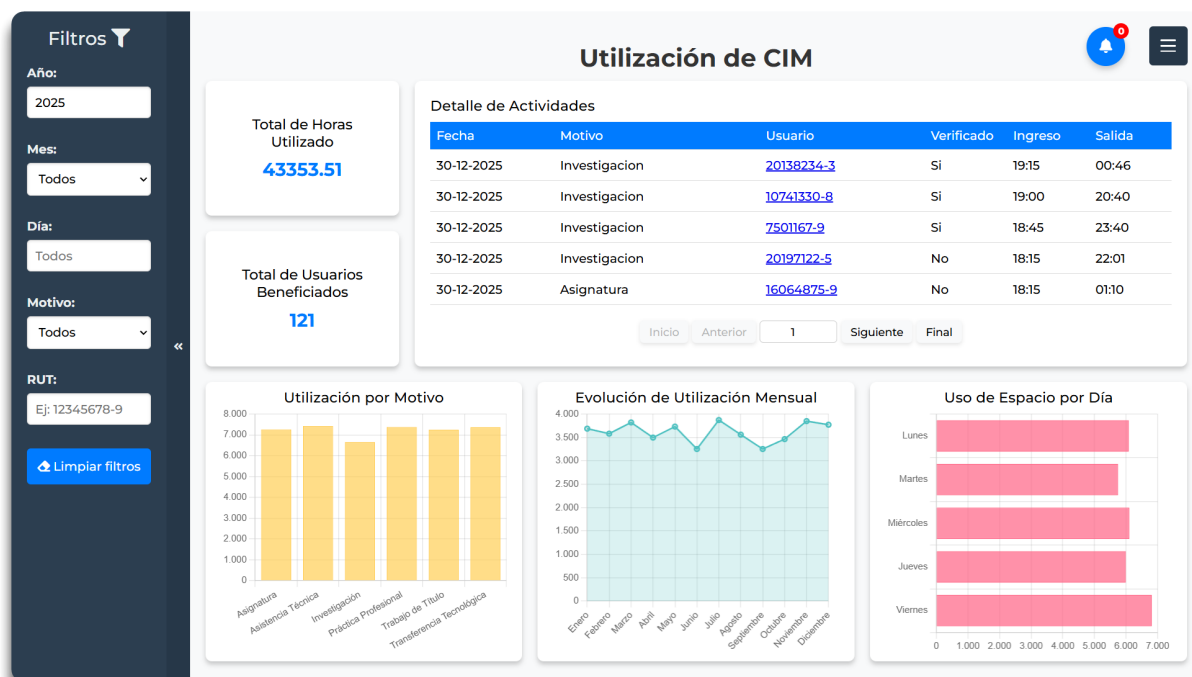


Figura 15: Interfaz del dashboard

- <http://{IP}:8000/profile?rut={RUT}>

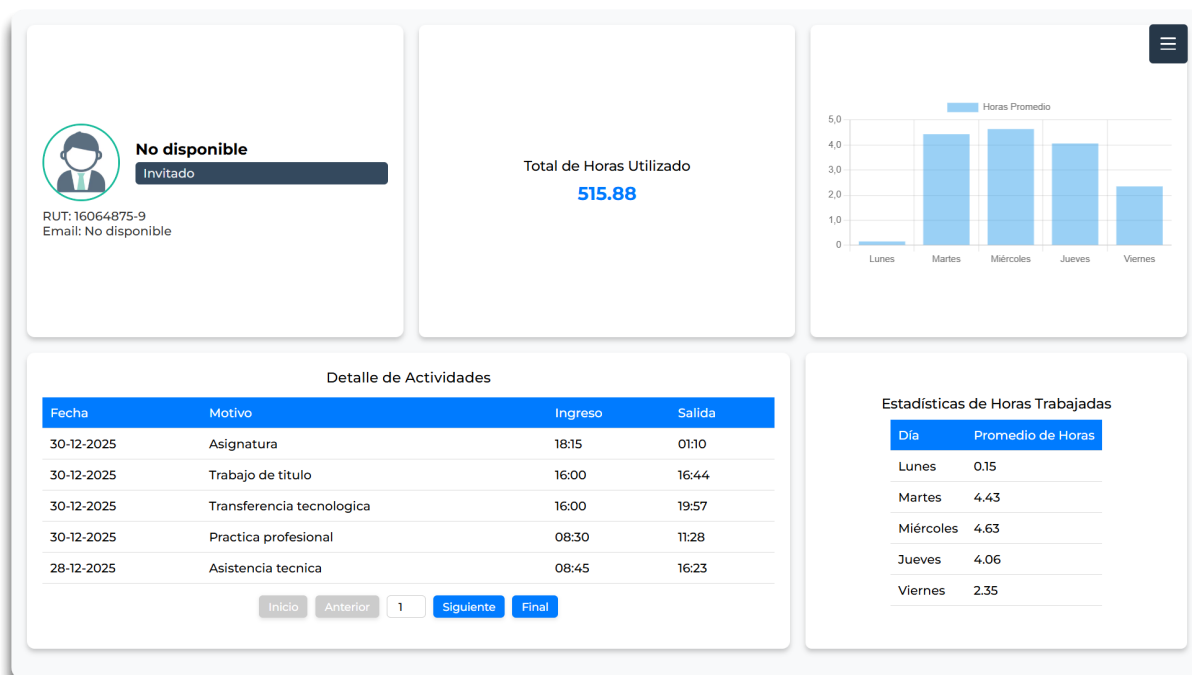
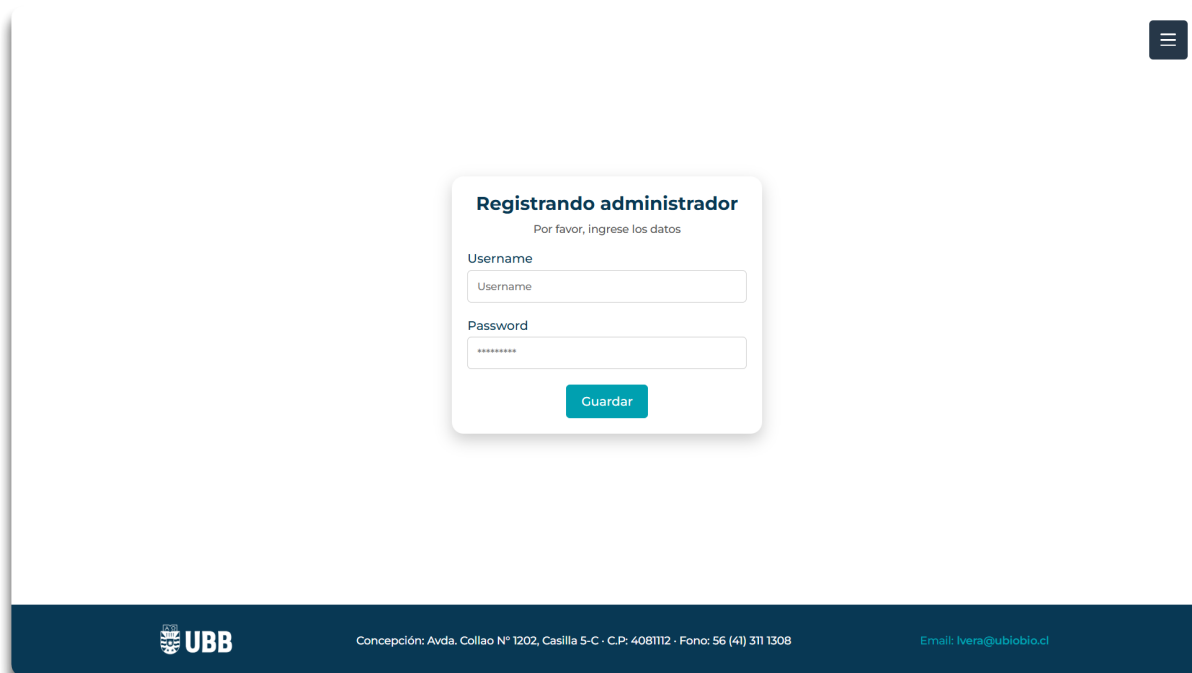


Figura 16: Interfaz del profile

- <http://{IP}:8000/register-admin>



Registrando administrador
Por favor, ingrese los datos

Username

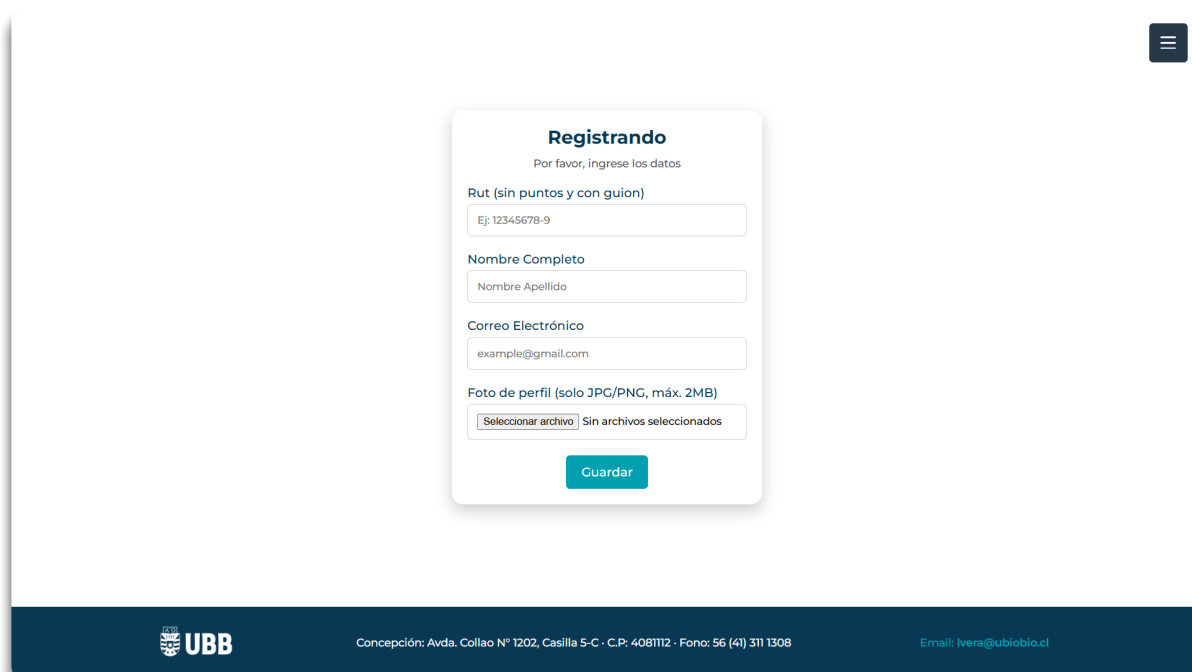
Password

Guardar

UBB Concepción: Avda. Collao N° 1202, Casilla 5-C · C.P: 4081112 · Fono: 56 (41) 311 1308 Email: lvera@ubiobio.cl

Figura 17: Interfaz del register admin

- <http://{IP}:8000/register-user>



Registrando
Por favor, ingrese los datos

Rut (sin puntos y con guion)

Nombre Completo

Correo Electrónico

Foto de perfil (solo JPG/PNG, máx. 2MB)
 Sin archivos seleccionados

Guardar

UBB Concepción: Avda. Collao N° 1202, Casilla 5-C · C.P: 4081112 · Fono: 56 (41) 311 1308 Email: lvera@ubiobio.cl

Figura 18: Interfaz del register user

■ Componente Navbar

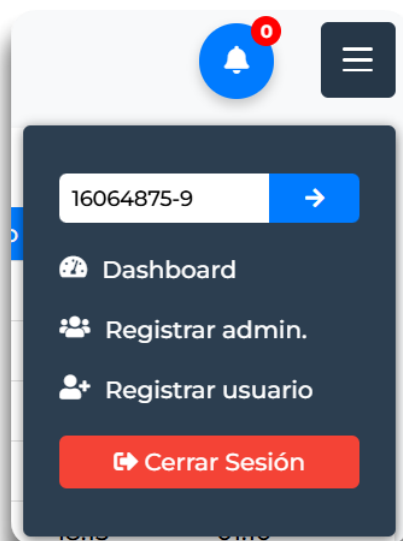


Figura 19: Componente Navbar

■ Componente Notify

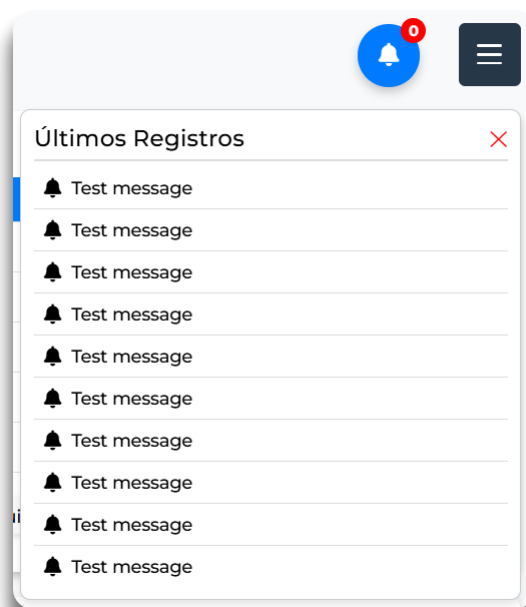


Figura 20: Componente Notify

Referencias

- [1] Raspberry Pi 4 Model B 8 GB RAM <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [2] Documentación sensor QR GM811 <https://robu.in/wp-content/uploads/2024/08/GM811.pdf>
- [3] Logitech HD Webcam C270 https://www.logitech.com/es-roam/products/webcams/c270-hd-webcam.960-000947.html?srsltid=AfmB0oqdKlo4lP1JIkMP6cvLw_Y6Kj5oKXIW-8gIEuyMUwLZRGhJMIwS
- [4] Generate Random Strings and Numbers <https://www.browserling.com/tools/random-string>